

WAR ROOM

A band of AI agents that runs your incident response

Coordinated entirely through Band — with a human holding the only key.

lablab.ai · Band of Agents Hackathon

github.com/TestStudent156/gpu-rental-mcp

THE PROBLEM

Incident response is a coordination problem

- The hard part isn't any single decision — it's the handoffs.
- Who investigates? Who's allowed to act? Who tells stakeholders?
- Who makes sure a bot doesn't roll back the wrong thing?
- Most AI-agent demos hard-wire the handoffs into a central orchestrator.
- That orchestrator IS the product — the agents are just function calls.

THE SOLUTION

Six participants. One Band room.

Detector	watches metrics, raises the alert	non-LLM
Commander	opens the incident, delegates, declares done	LLM
Diagnostician	pulls metrics / logs / deploys, finds cause	LLM
Remediator	proposes the fix; runs it only after approval	LLM
Comms	posts plain-English status updates	LLM
Bridge	mirrors room to dashboard, relays approval	non-LLM

ONE INCIDENT, START TO FINISH

Detect · Diagnose · Block · Approve · Heal

```
Detector      -> @commander    ALERT checkout error_rate 42%
Commander     -> @diagnostician investigate + @comms update
Diagnostician -> root cause: deploy dpl-104, NullPointerException
Remediator    -> rollback dpl-104      [ BLOCKED - need human ]
Commander     -> APPROVAL REQUESTED: rollback dpl-104 on checkout
               -- human clicks  APPROVE --
Server        -> rollback dpl-104      [ OK ] checkout healthy
Commander     -> INCIDENT RESOLVED
```

End-to-end in under a minute — every line is a real Band message.

OUR CORE BET

Band is the only channel

- No orchestrator. No message bus. No shared memory.
- Every handoff is a Band message with @mentions — agents act only when mentioned.
- The workflow emerges from who addresses whom, like a real on-call channel.
- Humans and bots are first-class peers — 2 of the 6 agents are plain Python.
- Even governance flows through the room: the human's decision is a Band message.

The human gate is unskippable

- Rollback / restart / scale are refused at the action layer until a human approves.
- No prompt can bypass it — it's enforced in code, not in instructions.
- Blocked attempts show on the timeline as visible proof of the gate.

```
[ BLOCKED ] rollback dpl-104 -- awaiting human approval
```

```
[ HUMAN   ] operator clicks APPROVE on the dashboard
```

```
[ OK      ] rollback dpl-104 -> checkout healthy
```

A live ops dashboard

- Service status board — health flips red to green as the incident resolves.
- Live timeline — the Band conversation and blocked actions, streaming in.
- APPROVE / REJECT — the single human action, front and center.
- MTTR — time from alert to resolved, ticking live.

How it fits together

BAND ROOM

commander · diagnostician · remediator · comms (talk only via @mentions)

Detector

raises the alert

Bridge (non-LLM)

mirrors the room to the dashboard · relays the human's approval back in

FastAPI server

MockOps sim · action-layer approval gate · dashboard (status, timeline, APPROVE, MTTR)

BUILT WITH

Tech stack

Band SDK

Python

Featherless AI

Qwen2.5-14B

FastAPI

SSE / polling

HTML / JS

Each agent is a plain LLM tool-loop — no agent framework.
1-unit model lets all four reasoning agents run in parallel.

War Room

A band of agents — and a human — running a real operation together.

Coordinated entirely through Band.

github.com/TestStudent156/gpu-rental-mcp